

An Adaptive Contention Window Backoff Scheme Differentiating Network Conditions Based on Deep Q-Learning Network

Zhi-Bin Zuo[✉], De-Min Wang[✉], Mi-Mi Ma[✉], Miao-Lei Deng, and Chun Wang

Abstract—In IEEE 802.11 networks, the Contention Window (CW) is a crucial parameter for wireless channel sharing among numerous stations, directly influencing overall network performance. In order to mitigate the performance degradation caused by the increasing number of stations in the network, we propose a novel adaptive CW backoff scheme, termed the ACWB-DQN algorithm. This algorithm leverages the Deep Q-Learning Network (DQN) to explore a CW threshold, which is utilized as a boundary to differentiate the network load circumstances and learn the best configurations for different network conditions. When stations transmit data frames, different CW optimization strategies are employed based on station transmission status and the CW threshold. This approach aims to enhance network performance by adjusting the CW to increase transmission efficiency when there are fewer competing stations, and lower collision probabilities when there are more competing stations. Simulation results indicate that this approach can optimize station CW, reduce network collision rates, maintain constant throughput and significantly enhance the performance of Wi-Fi networks by means of adjusting the CW threshold according to real-time network conditions.

Index Terms—IEEE 802.11, deep Q-learning network, wireless networks, deep reinforcement learning.

I. INTRODUCTION

OVER the past few years, with the rapid development of various smart terminal devices and mobile Internet of Things (IoT) services, the demands for wireless services in

terms of traffic and quality of service has been increasing [1]. Wi-Fi has become the primary means to deliver wireless services due to its high speed, flexible deployment, and cost-effectiveness. Sending and receiving data frames at the same time may lead to incomplete inputs, such as noise, hiccups, absent data, and other problems. The collision primarily occurs when a node accepts more than one frame at a time, which means both nodes (stations) clash because they have the same priority, resulting in no frames that are correctly delivered. As IEEE 802.11 WLAN becomes more widespread, the density of Wireless access devices per Access Point (AP) continues to increase. This has led to a rise in collision rates and a decline in throughput during data frame transmission by stations. In 2019, the IEEE released the latest Wi-Fi 6 standard [2] (802.11.ax). Wi-Fi 6 introduces cutting-edge modulation and encoding techniques, such as 1024-QAM (Quadrature Amplitude Modulation), designed to significantly enhance data transmission speeds. In the Wi-Fi 6 standard, CSMA/CA [3], [4], [5] (Carrier Sense Multiple Access with Collision Avoidance) is still used to avoid data transmission collisions between stations. The CSMA/CA protocol has been further optimized in Wi-Fi 6 by adding a trigger-based frame transmission mechanism (Trigger Frame). It enables stations to initiate data transmission without contention. Basic Service Set (BSS) coloring is also a new technique introduced in Wi-Fi 6. It is aimed at reducing interference between adjacent networks. By assigning a unique color identifier to each BSS, devices can better differentiate and identify different networks. Although these new technologies and improvements enhance the adaptability of the CSMA mechanism to high-density and high-load network environments, the current CSMA mechanism still struggles to accommodate changes in network conditions effectively, thereby failing to effectively reduce station collisions. In a network environment with dense devices, high contention for the channel leads to frequent collisions, resulting in significant degradation of network performance and an inability to provide sufficient bandwidth and a good user experience.

In the CSMA mechanism, nodes wait for a random time interval within a certain range before transmitting data frames. This range is referred to as the Contention Window (CW), and the random time interval is known as the Backoff Timer (BT). The BT for nodes is determined by the size of the CW. A large CW reduces the possibility that different nodes choose the same backoff time. This makes it harder for nodes to transmit

Received 8 September 2023; revised 22 May 2024; accepted 11 August 2025. Date of publication 20 August 2025; date of current version 5 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62276091 and Grant 42471518; in part by the Key Science and Technology Program of Henan province under Grant 252102210162; in part by the Open Fund from Research Platform of Grain Information Processing Center in Henan University of Technology under Grant KFJJ2024004; and in part by the Doctoral Research Grant Program of Henan University of Technology under Grant 2022BS005. The associate editor coordinating the review of this article and approving it for publication was M. J. Khabbaz. (Corresponding author: Mi-Mi Ma.)

Zhi-Bin Zuo is with the Key Laboratory of Grain Information Processing and Control, Ministry of Education, the Henan Key Laboratory of Grain Storage Information Intelligent Perception and Decision Making, and the Department of the School of Information Science and Engineering, Henan University of Technology, Zhengzhou 45001, China (e-mail: zzbzbin_2000@163.com).

De-Min Wang and Chun Wang are with the School of Information Science and Engineering, Henan University of Technology, Zhengzhou 45001, China (e-mail: wdemin2022@163.com; m17837698815@163.com).

Mi-Mi Ma and Miao-Lei Deng are with the Department of the School of Information Science and Engineering, Henan University of Technology, Zhengzhou 45001, China (e-mail: mamimi421@126.com; dmlei2003@163.com).

Digital Object Identifier 10.1109/TNSM.2025.3600861

data simultaneously, which in turn lessens station collisions. But it also means that stations have to wait longer time before they can transmit data. On the other hand, while a small CW guarantees stations shorter wait times, it is prone to selecting the same wait time for multi-nodes, which reduces channel utilization by the collisions caused during data transmission. Therefore, an effective CW backoff mechanism can reduce collisions and waiting delays by assigning suitable CW to stations for channel contention, thereby improving network performance.

Currently, ML (Machine Learning) methods are being employed for CW optimization [6], although their selection is limited by the characteristics of the problem. While analytical models may provide optimal CW values under certain assumptions and quasi-static conditions, these assumptions are seldom encountered in real-world networks. Another prevalent approach is gradient-based optimization, which ideally enables us to effortlessly determine the optimal value by calculating the derivative of the network throughput function concerning CW. Unfortunately, this isn't always practical. Reinforcement learning (RL) [7] is a machine learning method aimed at learning how to make a series of decisions to maximize the expected cumulative reward through interaction with the environment. In RL, an agent selects actions based on observations of the environment's state and learns from the rewards obtained from these actions. In wireless networks, RL can consider factors such as node addition or removal and changes in network load to adjust corresponding parameters and optimize the performance of wireless networks. In Vehicular Ad-Hoc Networks (VANET) [8], [9], [10], [11], Wireless Sensor Networks (WSN) [12], [13], Wireless Local Area Networks (WLAN) [14], [15], and other wireless networks, RL has been applied. The utilization of RL for optimizing traditional communication technologies has emerged as a significant trend in the advancement of wireless networks.

In this article, we propose a novel CW optimization scheme called Adaptive Contention Window Backoff algorithm with DQN (ACWB-DQN). ACWB-DQN scheme utilizes Deep Reinforcement Learning (DRL) [16] technology to discern the best effective CW threshold value, discerning the prevailing network load conditions. The scheme employs a novel backoff strategy specifically designed for different network loads to dynamically adjust the CW. The first strategy (Linear Increase, Exponential Decrease Strategy) is deployed when network load is low. Its objective is to minimize station waiting times and improve network transmission efficiency. Conversely, the second strategy (Exponential Increase, Linear Decrease Strategy) is applied in scenarios characterized by high network loads. This strategy prioritizes collision mitigation among stations, aiming to reduce collision rates and bolster overall network stability. ACWB-DQN utilizes DQN to adjust thresholds to adapt to dynamic changes in the network environment. Stations adjust their CW by selecting an appropriate backoff strategy based on the CW threshold, thereby reducing collision probability and waiting delays, and enhancing the performance of Wi-Fi networks.

Finally, the collision rate and throughput of this algorithm are analyzed by simulation experiments under different

station densities and make comparisons with the traditional CSMA/CA backoff algorithm, the CCOD-DQN algorithm, and the SETL-DQN algorithm. The experimental results demonstrate that the proposed ACWB-DQN scheme in this article achieves significantly lower collision rates compared to other schemes under varying network loads, while maintaining stable and high throughput. This validates the effectiveness of the novel CW adjustment strategy proposed in this article.

The contributions of this article are:

- The key factors affecting network throughput are identified and methods to improve network performance are proposed by modelling and further analyzing the IEEE 802.11 mechanism on the basis of Bianchi's two-dimensional Markov model [24]. Ultimately, it concludes that the key to enhancing network throughput lies in adaptively adjusting the CW values of stations based on different network conditions.
- Introducing the latest DRL technology into the CW adjustment process of IEEE 802.11. By utilizing DRL technology to perceive real-time network conditions, we can formulate appropriate CW adjustment strategies, allowing stations to maintain an optimal CW for channel access.
- A novel CW adjustment strategy is proposed, which employs different adjustment methods based on varying network conditions. Under low network load, the goal is to reduce the waiting time for stations, while under high network load, the primary objective is to reduce the collision rate. By maintaining a balance between collision rate and waiting time, this approach maximizes network throughput.
- The proposed CW optimization scheme is simulated and a prototype of this architecture is also implemented with Python code. It is compared with the other DRL-based CW scheme. The experimental results show that the proposed scheme effectively reduces collision rate and station waiting time, outperforming other algorithms.

The rest of this article is organized as follows: Section II provides background and related work. Section III describes modeling and analysis of IEEE 802.11 & applications of Deep Reinforcement Learning. Section IV analyzes and implements the proposed ACWB-DQN. Section V conducts simulation experiments and analysis on the ACWB-DQN scheme. Section VI discusses the conclusion.

II. BACKGROUND AND RELATED WORK

A. IEEE 802.11 DCF Mechanism

The Distributed Coordination Function (DCF) serves as the primary access mechanism within the IEEE 802.11 standard, facilitating shared access to the wireless medium among multiple stations with equal priority. DCF utilizes Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA) to regulate channel access and avoid collisions by first monitoring channel activity before transmitting data. The basic mechanism of CSMA/CA is illustrated in Fig. 1. STA represents the station with data ready for transmission, AP represents the wireless access point, $T_{backoff}$ represents the

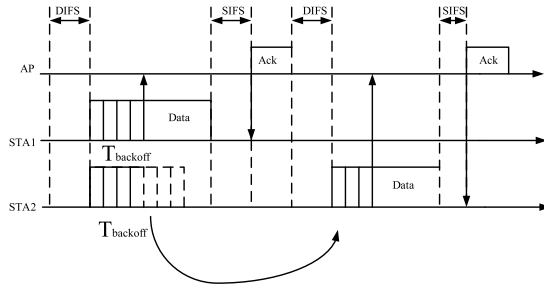


Fig. 1. Basic mechanism of CSMA/CA.

Backoff Timer (BT) value, which indicates the time a station needs to wait before sending data, and Data represents the data frame to be transmitted. The upward arrow represents the station sending data to the AP, while the downward arrow represents the AP sending data to the station. Before transmitting data, stations must monitor the channel. If the channel is idle after waiting for one Distributed Inter-Frame Space (DIFS) time, stations generate a BT value ($T_{backoff}$). The value is a multiple of the slot time, calculated as shown in (1). CW_{cur} represents the current CW size for the station; *random* is a function used to generate a random integer within a specified range, and σ represents the slot time.

$$T_{backoff} = \sigma \times \text{random}(0, CW_{cur} - 1) \quad (1)$$

Multiple stations are competing for channel access by generating BT values. During the backoff process, each station monitors the channel. If the channel remains idle after one time slot, the BT value of stations decreases by one slot. However, if there is data transmission ongoing in the channel, the BT of that station is frozen. The station will wait for a DIFS time, and if the channel is still idle, the station will resume the backoff process. When a station's BT value reaches zero, it gains access to the channel and starts transmitting data. However, if two or more station's BT value reach zero simultaneously, indicating that they both attempt to access the channel at the same time, a collision occurs. Stations that are involved in collisions utilize the Binary Exponential Backoff (BEB) algorithm, modify the size of the CW, and generate a BT value again before making another attempt to access the channel. When the Access Point (AP) receives data, it verifies the data using a checksum mechanism. Upon successful verification, the AP waits for a Short Interframe Space (SIFS) time and sends an ACK confirmation to notify the sender that the data has been successfully received.

The BEB backoff algorithm is a classical backoff algorithm used in CSMA mechanisms. The CW update strategy in BEB is shown in (2). CW_{min} represents the minimum CW value, and n denotes the number of retransmissions. In the IEEE 802.11 ax standard, CW_{min} is 15 slots, while CW_{max} is 1023 slots. During the initial channel contention, stations start with a default CW value of CW_{min} . In the case of data transmission collisions, the CW value exponentially adjusted, doubling each time. If collisions persist, the CW value will keep doubling until it reaches the CW_{max} . After the data is successfully transmitted, the CW value is reset to the CW_{min} .

In IEEE 802.11, the backoff algorithm is a significant factor influencing network performance. The goal of the backoff algorithm is to maintain the CW to accurately reflect the contention status of the channel near the node, to provide the node with the appropriate ability to preempt the channel. The value of CW directly affects the length of the generated delay time. When the value of CW is larger, the generated BT value is generally longer, and when the value is smaller, the generated BT value is generally shorter. Obviously, the smaller the value of the node's CW, the stronger its ability to preempt the channel; conversely, the weaker its ability to preempt the channel. In other words, the value of the CW reflects the node's access capability to the channel. The BEB algorithm has two drawbacks. Firstly, upon a successful interaction, the CW is reduced to its minimum value and fail to accurately reflect the contention status of the channel. Secondly, BEB can lead to significant unfairness. Successful nodes have their CW reset to the minimum value, whereas unsuccessful nodes maintain larger values. Therefore, successful nodes are more likely to win in subsequent competitions. This may result in certain nodes failing to successfully access the channel for an extended period, which leads to unfairness. This situation becomes particularly pronounced under high loads.

$$\begin{cases} CW_{cur} = CW_{min} & \text{success} \\ CW_{cur} = 2^n \times CW_{min} & \text{collision} \end{cases} \quad (2)$$

B. Related Work

The WLAN's performance is greatly limited by the Binary Exponential Back-off (BEB) algorithm in the CSMA mechanism. This method depends entirely on the success or failure of data transmission to change the CW blindly. Ali et al. [17] introduced a novel technique called Channel Observation-Based Scaled Backoff (COSB) to enhance the classic BEB backoff mechanism. The CW size of COSB is dynamically adjusted based on observations of the channel conditions. Cai et al. [18] introduced an eat-B algorithm that determines CW by considering the throughput of nodes. This enables nodes with higher throughput to perform the backoff process at a faster rate. These improvements have reduced collision rates and latency and network throughput is also enhanced. However, there are some problems. The stations need to undergo multiple collisions to adjust their contention window values to adapt to the current network conditions, and these algorithms can't dynamically adjust CW sizes based on station join or departure, leading to high collision rates in high-density stations.

With the emergence of RL, new ideas are provided to solve the above problems. Through interaction with the environment and agents, RL achieves real-time perception of network states. Ali et al. [19] enhanced the COSB approach introduced in [17] by employing RL techniques. The Q-Learning method was used to optimize the scaling backoff mechanism based on channel observation. Additionally, the intelligent QL-based Resource Allocation (iQRA) mechanism was presented for intelligent resource allocation. In the Enhanced Distributed Channel Access (EDCA) mechanism, the blind utilization of the BEB mechanism to avoid collisions among WLAN stations

sacrifices the performance of best-effort data (low-priority) traffic. Ali et al. [20] introduced a Machine Learning-supported EDCA (MEDCA) mechanism for QoS-supported MAC layer channel access in dense WLANs. This mechanism uses a Q-learning algorithm to estimate the density of the network and adaptively modifies the CW for each Access Category (AC). These studies employ Q-learning to forecast and adapt CW in order to improve network performance. Nevertheless, Q-learning is not without its limitations, including its relatively sluggish rate of convergence and limited ability to anticipate future outcomes. The main explanation for this is that Q-learning is an iterative value-update method that is unable to leverage past information when confronted with new circumstances.

Deep Q-network (DQN) is a value-based algorithm in DRL that primarily tackles certain challenges encountered in Q-learning. By leveraging the benefits of Q-learning and deep neural networks, DQN is effective in inferring the reward function more efficiently in unknown states and promotes decision-making capabilities. When compared to basic Q-learning, DQN demonstrates quicker convergence and an enhanced ability to anticipate future outcomes. Hence, the DQN technique is employed for CW optimization in intricate network circumstances as well. In the face of the proliferation of stations in Wi-Fi networks, the traditional 802.11 standard suffers from insufficient scalability to maintain stable throughput. To solve this problem, Wydmański and Szott [21] proposed the Centralized Contention Window Optimization with DRL (CCOD) scheme, which increases network throughput by accurately predicting the optimal CW values under different network conditions using DQN. However, the CCOD-DQN scheme utilizes the traditional Binary Exponential Backoff (BEB) algorithm to predict the optimal CW values. The BEB algorithm cannot dynamically adapt to changing network conditions. In the case of a heavy network load and a large number of competing stations, a single successful data transmission will shrink the CW value to a minimum, thus increasing the probability of station collision. In addition, the station's CW value needs to start from the minimum value and gradually find a suitable CW value by doubling the CW value after a collision, which may require the station to wait for a long time to successfully send a data frame. Thus, large transmission waiting delays and high collision rates are possible outcomes of the CCOD-DQN algorithm, especially when there are significant high-load network environments. To overcome these problems of the BEB algorithm, Ke et al. [22] proposed the Smart Exponential-Threshold-Linear (SETL) backoff algorithm. The algorithm discerns network load conditions by the CW threshold. It determines the adjustment strategy of CW after each data transmission by setting a threshold value. When the CW value is smaller than the CW threshold value, the network is considered to be in a low-load condition. The station CW value is adjusted exponentially to reduce the collision probability; on the contrary, if the CW value is larger than the CW threshold value, the network is considered to be in a high-load condition. The station CW value is adjusted linearly to avoid a larger transmission delay. Compared with the BEB algorithm, the

SETL algorithm has higher network throughput and lower collision rates under different network loads. However, the SETL backoff algorithm does not specify the procedure for determining appropriate CW thresholds under varying network loads. Ke and Astuti [23] used DRL techniques and proposed a Smart Exponential-Threshold-Linear with DQN (SETL-DQN) scheme to dynamically determine the CW threshold in the SETL backoff algorithm. Nevertheless, the SETL backoff algorithm utilized by SETL-DQN does exhibit certain limitations. In low-load networks, the exponential adjustment of CW values by the SETL backoff algorithm reduces collision probability. However, collisions are less likely in such networks, and rapidly increasing CW for stations via exponential growth may lead to longer waiting delays. On the other hand, in high-load networks, linear adjustment of CW values reduces transmission delays. Nevertheless, in heavily loaded networks, collisions are more probable. The slow increase in the CW value may result in a higher probability of collisions after a collision, thereby affecting network performance. Thus, the SETL-DQN scheme relying solely on either exponential or linear backoff strategies may not effectively accommodate current network conditions. Below the CW threshold, employing the exponential strategy can result in extended waiting times for stations. Conversely, above the CW threshold, utilizing the linear strategy may escalate collision rates. While these schemes employing DRL technology exhibit a notable reduction in collision rates, there remains potential for improvement due to the limitations of the chosen backoff algorithms.

To further improve the performance of Wi-Fi networks, the issues like heightened collision rates and longer transmission delays resulting from shortcomings should be solved in existing backoff algorithms. The ACWB-DQN scheme proposed in this article distinguishes the current network load condition by using a CW threshold and sets different CW optimization strategies according to the threshold and transmission status of the station. When the CW value is less than the threshold, the ACWB-DQN algorithm adopts the first strategy (*Linear Increase, Exponential Decrease Strategy*) to keep the CW value of the station at a smaller value, thereby reducing the waiting delay and improving the transmission efficiency. When the CW value is greater than the threshold value, the algorithm adopts the second strategy (*Exponential Increase, Linear Decrease Strategy*) so that the CW value of the station can maintain a larger value, thus reducing the possibility of collision and improving the throughput of the network.

III. MODELING AND ANALYSIS OF IEEE 802.11 AND APPLICATIONS OF DEEP REINFORCEMENT LEARNING

A. Modeling and Analysis of CSMA/CA Mechanism

The performance of the CSMA/CA protocol directly impacts the performance of IEEE 802.11 networks. Within CSMA/CA, there are numerous parameters that influence performance. To theoretically understand the relationship between these parameters and the performance of the CSMA/CA protocol, this article utilizes a widely adopted theoretical analysis method for CSMA/CA protocols, employing Bianchi's [24] two-dimensional Markov model. By analyzing

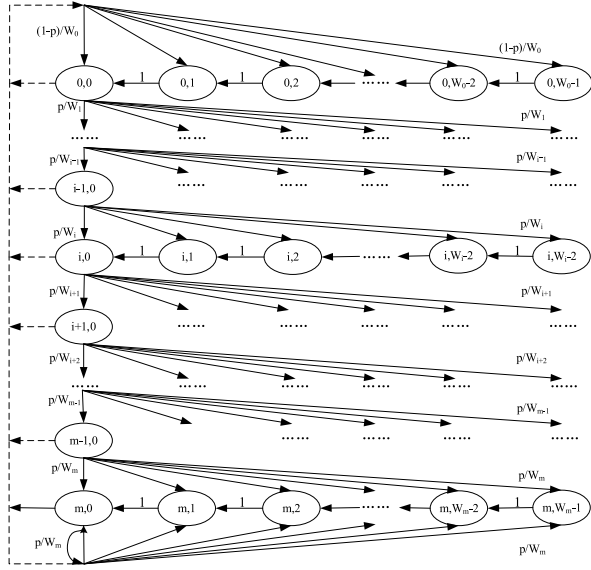


Fig. 2. Markov chain model of the CSMA/CA.

and deriving the relationship equations between CSMA/CA protocol performance and relevant parameters in the network, this article aims to identify the key factors affecting the performance of the CSMA mechanism.

1) *Modeling of CSMA/CA Mechanism*: In the establishment of the Markov model, the following assumptions are made: (1) the bit error rate during channel transmission is zero; (2) there are no hidden nodes; (3) the number of stations is fixed; (4) each station has data frames waiting to be transmitted at any given time; (5) the probability of collision for each station is constant and denoted by p ; (6) there is no upper limit on the number of retransmissions for each station.

In the Markov model, two random processes, denoted as $s(t)$ and $b(t)$, are defined to represent two attributes of the station's state. $s(t)$ represents the total number of backoffs generated by the station up to time t , ranging from 0 to m , where m represents the maximum backoff count. $b(t)$ represents the value of the BT for the station at time t , with t and $t+1$ representing the starting times of two consecutive time slots. The BT decrements by one at the beginning of each time slot. When studying the changes in station states, the focus is primarily on the starting time of each time slot during the countdown of the BT, while ignoring the process of the backoff timer being frozen. In this way, the two-dimensional random process $\{s(t), b(t)\}$ forms a two-dimensional discrete Markov chain. With regard to the process of stations using the CSMA/CA backoff algorithm for communication, a two-dimensional Markov model as, illustrated in Fig. 2, is established.

The variables defined in this model hold the following significance: (i, k) , where $i \in [0, m]$, signifies the current number of backoff counts at the station, with m denoting the maximum allowed backoff attempts. Initially, when the station intends to transmit a data frame, i equals 0. $k \in [0, W_m - 1]$, which represents the backoff timer (BT). $W_i = 2^i W$, $CW_{min} = W$, $CW_{max} = 2^m W$. p denotes the probability of communication

collision, while $1 - p$ represents the probability of successful communication.

The directional flow within this model indicates: The transition probability from (i, k) to $(i, k-1)$ is 1, indicating that from time t to $t+1$, the BT consistently decrements by 1, ultimately reaching the state $(i, 0)$. Communication initiation at the station is only permissible in state $(i, 0)$, ensuring an idle channel. During communication attempts in state $(i, 0)$, two contrasting outcomes may arise: a collision with probability p or a successful transmission with probability $1-p$. Following a communication collision, the station increases its backoff count by 1, transitioning from state $(i, 0)$ to $(i+1, k)$. Additionally, the CW undergoes exponential growth until reaching CW_{max} . Upon successful communication, the station resets its backoff count to 0, transitioning from state $(i, 0)$ to $(0, k)$.

In a Markov model diagram, downward arrows indicate collisions in station communication, where the backoff counter of station i increments by 1 and a new BT value (k) is chosen. The next state will be any state in the next row, assuming the next row is the $i+1$ row, with a transition probability of p/W_{i+1} . If the backoff counter i has already reached m , then the value of i remains unchanged, and its next state will still be any state in the last row, with a transition probability of p/W_m . Crosswise arrows in each row indicate the decrement of the BT value (k), and until the backoff timer decreases to $k=0$, only this crosswise change occurs, with a transition probability of 1. In this model, the single-step transition probability equation can be summarized as:

$$p(i, k | i, k+1) = 1, k \in [0, W_i - 2], i \in [0, m] \quad (3)$$

$$p(0, k | i, 0) = \frac{1-p}{W_0}, k \in [0, m], i \in [0, m] \quad (4)$$

$$p(i, k | i-1, 0) = \frac{p}{W_i}, k \in [0, W_i - 1], i \in [1, m] \quad (5)$$

$$p(m, k | m, 0) = \frac{p}{W_m}, k \in [0, W_m - 1] \quad (6)$$

Equation (3) explains that the BT value k decreases by 1 when the next time slot arrives. Equation (4) the station's backoff count is reset to 0, and a random backoff time is chosen between $(0, W_0 - 1)$ after successfully transmitting a data frame. Equation (5) explains that when the backoff count is $i-1$, if the station fails to transmit a data frame, the backoff count is incremented by 1, and a new BT (k) is randomly selected between $(0, W_i - 1)$. Equation (6) clarifies that after the backoff count reaches m , even if the data frame transmission fails, the backoff count no longer increases. Instead, it remains at m , and only the BT (k) needs to be randomly chosen again between $(0, W_m - 1)$.

p is used to represent the probabilities of each state appearing in the Markov chain. Let $b_{i,k} = \lim_{t \rightarrow \infty} p\{s(t) = i, b(t) = k\}$ $i \in (0, m)$, $k \in (W_i - 1)$ be the limiting distribution probability of the Markov chain. It is easy to obtain (7).

$$\begin{cases} b_{i,0} = p^i \times b_{0,0} & i \in [0, m] \\ b_{i,k} = \frac{W_i - k}{W_i} \times b_{i,0} & i \in [0, m] \\ b_{m,0} = \frac{p}{1-p} \times b_{0,0} \end{cases} \quad (7)$$

Because the sum of probabilities for all stable states must equal 1, we can obtain (8).

$$\begin{aligned}
 1 &= \sum_{i=0}^m \sum_{k=0}^{W_i-1} b_{i,k} \\
 &= \sum_{i=0}^m b_{i,0} \frac{W_i + 1}{2} \\
 &= \frac{b_{0,0}}{2} \left[W_0 \left(\sum_{i=0}^{m-1} (2p)^i + \frac{(2p)^m}{1-p} \right) + \frac{1}{1-p} \right] \quad (8)
 \end{aligned}$$

Therefore, the value of $b_{0,0}$ can be calculated, as shown in (9).

$$b_{0,0} = \frac{2(1-2p)(1-p)}{(1-2p)(W+1) + p \times W \times (1-(2p)^m)} \quad (9)$$

Let τ denote the probability that a station is in communication at any time. Since the station's transmission can only occur when the backoff timer decreases to zero, τ is the sum of the steady-state probabilities for all states where $k=0$. Therefore, the calculation of τ is as shown in (10).

$$\begin{aligned}
 \tau &= \sum_{i=0}^m b_{i,0} = \frac{b_{0,0}}{1-p} \\
 &= \frac{2(1-2p)}{(1-2p)(W+1) + p \times W \times (1-(2p)^m)} \quad (10)
 \end{aligned}$$

From equation (10), it can be understood that if both W and m are considered constants, then the probability τ of data being transmitted is a function of the collision probability p . However, the collision probability p can also be expressed as a function of the data transmission probability τ , as shown in (11).

$$p = 1 - (1 - \tau)^{n-1} \quad (11)$$

With the number of stations n known, (9) and (10) represent a system of two equations in two unknowns, τ and p . Hence, the communication probability τ is indeed a function of the initial CW size W and the number of backoff counts m .

2) *Analysis of CSMA/CA Mechanism:* In the previous section, the data frame transmission probability τ was calculated using the Markov model. This section will analyze the CSMA/CA model to investigate the factors influencing system throughput and derive the expression for system throughput.

S represents the system's saturated throughput. P_{tr} denotes the probability that at least one station can communicate within a unit of time. P_s represents the probability of successful communication under the condition that at least one station can communicate within a unit of time.

P_{tr} can be easily obtained through (11), as shown in (12).

$$P_{tr} = 1 - (1 - \tau)^n \quad (12)$$

Because when only one station is communicating, there will definitely be no collisions. Since at that moment all n stations might be able to communicate, the probability of successful communication is $n \times \tau \times (1 - \tau)^{n-1}$. Therefore, we can obtain the calculation formula for P_s , as shown in equation (13).

$$P_s = \frac{n \times \tau \times (1 - \tau)^{n-1}}{P_{tr}} = \frac{n \times \tau \times (1 - \tau)^{n-1}}{1 - (1 - \tau)^n} \quad (13)$$

During the communication process at the station, within a unit of time, only one of the following three situations will occur: (1) Successful communication with a probability of $P_{tr} \times P_s$; (2) Communication collision with a probability of $P_{tr} \times (1 - P_s)$; (3) No communication demand with a probability of $1 - P_{tr}$.

Assuming that the average length of each frame is $E[p]$, the calculation formula for the saturated throughput S can be obtained, as shown in (14). Where T_s is the average duration of channel occupancy under the condition of successful communication; T_c is the average duration of channel occupancy under the condition of communication collision; σ is a slot duration.

$$S = \frac{P_s \times P_{tr} \times E[p]}{(1 - P_{tr}) \times \sigma + P_{tr} \times P_s \times T_s + P_{tr} \times (1 - P_s) \times T_c} \quad (14)$$

Let the header be denoted as $H = PHY_{hdr} + MAC_{hdr}$, δ represent the transmission delay. Under the basic mechanism, it can obtain the values of T_s and T_c as shown in equation (15).

$$\begin{cases} T_s = H + E[p] + SIFS + \delta + ACK + DIFS + \delta \\ T_c = H + E[p] + DIFS + \delta \end{cases} \quad (15)$$

Next, let's discuss how to maximize S , and equation (14) can be simplified to:

$$S = \frac{E[p]}{T_s - T_c + \frac{\sigma \times (1 - P_{tr})}{P_s}} \quad (16)$$

In (16), when T_s , T_c , $E[p]$, and σ are fixed, maximizing S is equivalent to maximizing equation (17), which means keeping the denominator in (16) as small as possible. In (17), $T'_c = \frac{T_c}{\sigma}$.

$$\frac{P_s}{\frac{(1 - P_{tr})}{P_{tr}} + \frac{T_c}{\sigma}} = \frac{n \times \tau \times (1 - \tau)^{n-1}}{T'_c - (1 - \tau)^n (T'_c - 1)} \quad (17)$$

In (17), the network scale (number of stations n) and T'_c are parameters determined by the network itself, and users cannot adjust their values. Therefore, to maximize the system's saturated throughput S , one only needs to adjust τ . From (10) and (11), we can infer that τ is a function of W and m , where W and m are adjustable parameters. For a network, as long as there are reasonable initial CW sizes W and backoff counts m , it's possible to effectively increase the value of S . Setting m is equivalent to setting CW_{max} . Only when the station backoff count reaches m does m come into play. When the load is light, the backoff count rarely reaches m . When the load is heavy, m starts to play a role, but by then the collision rate is already high, so adjusting the value of m does not have a significant effect.

In the CSMA mechanism, if a station successfully sends data frame, its CW will reset to W . If the transmission fails, the CW for the station is set $2^i W$. The value of W determines the size of the CW for the station during the data transmission process. However, the initial CW value W remains constant and does not adapt to changes in network conditions. Therefore, in a specific network, only one data

frame transmission probability τ can be associated. However, in reality, network conditions are dynamically changing. When network load changes, τ should be adjusted accordingly to adapt to different network conditions. Thus, to change τ , it is necessary to adaptively adjust the value of W , thereby dynamically changing the CW size for the station after each data transmission.

DRL techniques excel at perceiving the real-time network state. By treating the network state as the environment, station actions as actions, and network performance metrics as reward signals, stations can intelligently adapt their behavior based on the current network conditions. This enables stations to dynamically learn and adapt to the constantly changing network environment, thus effectively managing contention window sizes and enhancing the overall performance and efficiency of the network. Therefore, the adaptive backoff algorithm based on DRL can perceive the network condition and adjust the CW of stations accordingly, aiding stations in competing for channel resources more effectively and reducing collisions and transmission delays, thereby enhancing network performance.

B. Definition of Partially Observable Markov Decision Process

Markov Decision Process (MDP) is a mathematical framework for modeling interactive decision problems with environmental states, agent decisions, and rewards. In a standard MDP, the agent can fully observe the current state and make decisions accordingly. However, in real network environments, the network state is not easily accessible due to the inherent uncertainty and randomness of the wireless channel. Partially Observable Markov Decision Process (POMDP) is an extension of MDP in which the agent does not directly observe the complete state of the environment. Instead, it infers potential global state distributions from observed local information or partial observations. In POMDP, an agent needs to consider its current observations as well as its previous observations and action history to make decisions.

In 802.11 network, it is assumed that there is an AP and n stations. When the AP considers adjusting the current CW threshold value, POMDP can be used to describe the CW threshold adjustment process. In this scenario, the agent is located at the AP, which can determine the appropriate CW threshold value by observing the network condition and broadcasting the CW threshold value to all the stations via beacon frames. The POMDP can be defined as a 7-tuple $(S, A, T, R, \Omega, O, \gamma)$ as follows:

S is the state space set ($s \in S$), which represents the current situation of all nodes in the network. In POMDP, agent cannot know their environment's state accurately, but they can get incomplete information about the state through observation and thus rely on the history of observation and actions to make decisions.

A is the action space set ($a \in A$) and represents the possible actions to be selected in the current state and it is used to adjust the size of the CW threshold value (CW_{thr}). The definition of the CW_{thr} is shown in (18), where $a \in [0,7]$, and thus, the

CW_{thr} ranges from 128 to 1024.

$$CW_{thr} = 2^7 \times (1 + a) \quad (18)$$

T is the state transition probability, representing the probability distribution of performing an action in the current state (s) and transferring to another state (s').

R is the reward function, which represents the reward received for reaching the next state (s') after executing an action in the current state (s). In Wi-Fi, throughput is an important metric for measuring network performance, hence selecting throughput as the reward function. Since in DRL the reward should be between 0 and 1, the throughput needs to be normalized by dividing the successfully received bits by the expected maximum bits.

Ω is the observation space set ($o \in \Omega$), which represents all possible observations the agent can perceive, and O is the set of conditional observation probabilities. When the agent takes an action a , it receives an observation o , which depends on the new state (s') of the environment with probability $O(o|s', a)$. In this scheme, the historical collision rate ($H(P_{col})$) of the network is defined as the observation o , where P_{col} denotes the probability of unsuccessful transmission (collision rate), which is defined as shown in (19), where N_t is the total number of frames sent by the station and N_r is the number of frames received correctly. In 802.11 network, the AP can know the total number of frames N_t sent by the stations and can obtain the number of successfully received frames N_r to calculate P_{col} . The $H(P_{col})$ is the difference between the collision rate (P_{col}) of the previous state and the current state. By observing $H(P_{col})$, we can know whether the frame collision rate increases or decreases at each time point. If the collision rate increases, the parameters can be adjusted and optimized again.

$$P_{col} = (N_t - N_r)/N_t \quad (19)$$

$\gamma \in [0,1]$ is the discount factor, which is used to measure how much the agent values future rewards. When γ tends to 1, the agent considers future rewards to be more important, and if γ tends to 0, the agent focuses more on current rewards.

C. Deep Reinforcement Learning

DRL is a ML method that combines DL and RL to learn the optimal strategy through the interaction of the agent with the environment [25]. The agent selects an action by observing the state of the environment and adjusts the strategy based on the feedback (reward or punishment) from the environment. The goal of DRL is to find the optimal strategy by maximizing the cumulative rewards so that the agent is able to make the optimal decision in various states. Q-learning is an unsupervised learning technique for solving reinforcement learning problems. It learns the value function of an action (Q-value function) by interacting with the environment, Q-value represents the expected cumulative reward gained by choosing an action in a given state. The basic idea of Q-learning is to use a Q-table to store the Q-value corresponding to each state and action, and to approximate the optimal Q-value by iterative updating. However, the disadvantage of Q-learning is

that it is slow to converge, especially when the state space is large, and it does only rely on previous experience and poorly handles new states encountered [26]. In a Wi-Fi environment, the number of possible states and actions makes simple Q-learning not feasible. The DQN algorithm is an approach to improve Q-learning using deep neural networks. DQN approximates the Q-function by using a neural network that takes the states as inputs and outputs the Q values corresponding to each action. This allows to handle the situations with large state spaces and the ability to learn better strategies from prior experience. In addition, DQN introduces the concepts of experience replay and target networks to further improve the efficiency and stability of learning [27]. In DQN, the parameters of the neural network are denoted as θ , and the loss function $L(\theta)$ is constructed using the Q-learning algorithm. By minimizing the loss function, the weight parameters of the neural network can be updated to gradually approximate the optimal Q function. The basic updating formula of DQN is shown in (20), where $Q(s, a; \theta)$ denotes the estimated Q value of the current neural network for state (s) and action (a), and $\max Q_{a'}(s', a'; \theta')$ denotes the estimated Q value of the target network for the next state (s') and the next action (a'), r denotes the reward given by the environment, and γ is a discount factor used to balance the importance of immediate and future rewards.

$$L(\theta) = E \left[\left(r + \gamma \times \max Q_{a'}(s', a'; \theta') - Q(s, a; \theta) \right)^2 \right] \quad (20)$$

IV. PROPOSED ACWB-DQN ALGORITHM

A. ACWB-DQN Algorithm

The ACWB-DQN algorithm is an adaptive CW backoff algorithm that employs DRL techniques to adjust the CW, aiming to optimize transmission efficiency and throughput in Wi-Fi networks. It is divided into two main processes: the reinforcement learning process and the channel access process. The reinforcement learning process explores the CW_{thr} suitable for the real-time network condition by utilizing the DRL technique. In the channel access process, the scheme uses this CW_{thr} for the communication process of stations and executes an appropriate backoff strategy based on the CW_{thr} ensuring that station's CW values are suited to the real-time network conditions. This adaptive approach helps in avoiding collisions and reducing waiting delays, thereby enhancing transmission efficiency. Through the continuous interaction of these two processes, ACWB-DQN is able to implement an adaptive CW optimization strategy to improve the performance of the Wi-Fi network in the process of continuous learning and adjustment.

1) *The Reinforcement Learning Process:* The reinforcement learning process mainly predicts and adjusts the CW_{thr} by utilizing the DRL technique. The current network condition is first used as a state representation. Since the network condition is not easily accessible directly in wireless networks, observation values (historical collision rates) are used as a representation of the network state so that the DQN can perceive the current network condition. At each time step, the DQN perceives the current network state based on the current observations and selects an action to adjust the size of the

CW_{thr} . The action can increase, decrease, or keep the size of the CW_{thr} constant to seek better network performance. The DQN guides the decision-making process based on the reward function. In this case, the reward function is the normalized throughput obtained by applying the CW_{thr} to a transmission period. Normalized throughput is a measure of the number of bits transmitted in a given period of time and is used to measure how good the current network performance is. The DQN then updates the action value function (Q-value function) to optimize the adjustment strategy of the CW_{thr} based on the selected action and the corresponding reward. Through continuous state observation, action selection, and reward feedback, DQN can gradually learn an optimal CW_{thr} adjustment strategy to maximize the reward function. Through the above learning process, the DQN can autonomously adjust the size of the CW_{thr} to improve the transmission efficiency and throughput of the Wi-Fi network and adapt to the dynamically changing network environment.

2) *The Channel Access Process:* The channel access process is mainly done by setting different CW optimization strategies to dynamically adjust the CW value according to the CW_{thr} value. The CW optimization strategy in ACWB-DQN is shown in (21). IEEE 802.11 specifies that CW_{min} is 15 and the CW_{max} is 1023. At the initial time, the station selects CW_{min} as the current CW value (CW_{cur}). When stations contend for a channel, the station compares CW_{cur} with the current CW_{thr} of the network, and based on the result of the comparison, different adjustment strategies are adopted.

$$\begin{cases} CW_{cur} > CW_{thr} & \begin{cases} CW_{cur} = CW_{cur} - 32 & \text{success} \\ CW_{cur} = CW_{cur} \times 2 & \text{collision} \end{cases} \\ CW_{cur} \leq CW_{thr} & \begin{cases} CW_{cur} = \frac{CW_{cur}}{2} & \text{success} \\ CW_{cur} = CW_{cur} + 32 & \text{collision} \end{cases} \end{cases} \quad (21)$$

a) Strategy 1: Linear Increase, Exponential Decrease Strategy

If $CW_{cur} \leq CW_{thr}$, ACWB-DQN adopts the adjustment strategy of linear increase and exponential decrease. In the case where the data frames sent by the station fail, the CW_{cur} is the current CW value plus 32, where 32 is the optimal CW growth value selected after many experiments; in the case of a station's successful transmission data frame, the CW_{cur} is the current CW value halved. A CW value below the CW threshold value indicates that there are fewer contending stations in the current network and that the network load is less severe. In this network scenario, if the frame sending fails, the algorithm uses a linear approach to slowly increase the CW value to avoid further collisions and reduce the transmission delay; if the frame sending succeeds, the ACWB-DQN uses an exponential approach to quickly reduce the CW value to reduce the waiting delay and improve the transmission efficiency.

b) Strategy 2: Exponential Increase, Linear Decrease Strategy

If $CW_{cur} > CW_{thr}$, ACWB-DQN adopted the adjustment strategy of exponential increase and linear decrease. When the station's data frame transmission

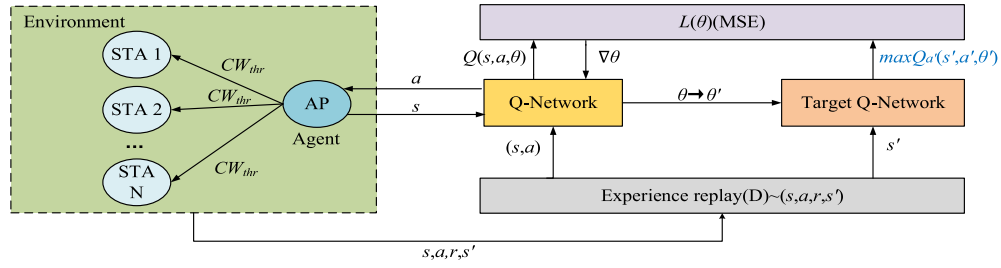


Fig. 3. Overall model diagram.

fails, the CW_{cur} will be doubled. If the station successfully transmits a data frame, the CW_{cur} is the current CW value minus 32. When the CW value is above the threshold, it indicates that there are many competing stations and a heavy network load. In this network case, if the frame transmission fails, the algorithm uses an exponential way to rapidly increase the value of CW to reduce the competition with other stations, thus reducing the possibility of collision. If the frame is successfully sent, the ACWB-DQN algorithm slowly linearly reduces the CW value to gradually narrow the CW, thereby avoiding high collision rate possibilities and improving transmission efficiency.

After the CW value of the stations is adjusted, it needs to be confirmed to prevent the CW value from exceeding the range specified by IEEE 802.11. If the adjusted CW is less than the CW_{min} , adjust it to CW_{min} ; if it is greater than the CW_{max} , adjust it to CW_{max} , as shown in (22).

$$\begin{cases} CW_{cur} = CW_{max} & CW_{cur} > CW_{max} \\ CW_{cur} = CW_{min} & CW_{cur} < CW_{min} \end{cases} \quad (22)$$

B. Implementation of the ACWB-DQN Schemes

The implementation of the ACWB-DQN scheme is mainly divided into two parts: the implementation of the reinforcement learning process and the implementation of the channel access process. In the implementation of the reinforcement learning process, the system periodically deposits a large amount of historical experience data into the experience pool, where each piece of data is stored in the form of (s, a, r, s') . In each time step, the system periodically extracts batch data from the experience pool for training, updates the action value function according to the selected action and the corresponding reward, and gradually to explore the CW_{thr} that best suits the current network environment. In the implementation of the channel access process, the AP periodically obtains the CW_{thr} from the DQN model and broadcasts it to all the stations (STAs). After receiving the CW_{thr} , the STAs dynamically adjust the CW to contend for the channel based on the current CW_{thr} , selecting the most suitable CW optimization strategy according to the current network conditions. AP evaluates the network performance of the current CW_{thr} for this period and calculates the reward (r) and the next state (s'). The experience samples (s, a, r, s') form this period are then stored in the experience pool in the reinforcement

learning process. The schematic of the model is shown in Fig. 3.

1) *Implementation of the Reinforcement Learning Process:* The reinforcement learning process primarily utilizes DRL techniques to predict and adjust the CW threshold (CW_{thr}). By employing POMDP, the agent (AP) can effectively handle situations with incomplete observations, making appropriate decisions based on perceived network conditions to determine the optimal CW_{thr} . Once CW_{thr} is determined, the channel access process begins. The AP broadcasts CW_{thr} to all connected stations, initiating channel contention and data transmission. At the end of the transmission period, the agent (AP) gets the normalized throughput and the historical collision rate as a reward r and an observation o , and determines the new state (s') of the environment. The agent (AP) stores the data of each time step to form the experience replay buffer, and each experience contains a quaternion (s, a, r, s') consisting of the current state, the executed action, the obtained reward, and the next state. The DQN model uses the historical collision rate and normalized throughput obtained from the current CW_{thr} to adjust the action selection strategy to re-generate a new CW_{thr} . APs and STAs re-engage the channel access process based on the new CW_{thr} to enter the next transmission period. Continuously repeat the above process. Finally, the DQN model updates the network model after a certain number of steps to minimize the error between the model's predicted value and the target value. The model's performance is continuously optimized through iterative training and tuning of network parameters to predict the best CW_{thr} for optimal network performance.

The details of the DQN algorithm are shown in Algorithm 1. Firstly, the experience buffer D and some key parameters are initialized. The weight parameters of the current network and the target network should be consistent, i.e., $\theta' = \theta$. After the initialization is completed, the training starts to be executed, and then each time step is executed. For each time step, the main network (Q-Network) uses the ϵ -greedy method to select an action a in the current state. The agent generates CW_{thr} based on the selected action. AP broadcasts CW_{thr} and collects the network performance obtained with this CW_{thr} . Stations send data and access the channel by adjusting the CW value according to the CW_{thr} . After one transmission period, the AP evaluates the network performance for that period and calculates normalized throughput and historical collision rate. The agent uses the historical collision rate as o , throughput as r , the next state s' through the observation

Algorithm 1: ACWB-DQN algorithm

Define CW_{thr} : as contention window threshold value.
 Define s : as the current state, a : as the chosen action, r : as the obtained reward, s' : as the state after performing action
 Define $Episode_{max}$: as the total number of episodes for training,
 $step_{max}$: as the maximum number of steps per episode during training,
 C : as the parameter determining the frequency of updating the target network parameters.

- 1 **Initialize** *Initialize current network* $Q(s, a; \theta)$ with random network parameters θ ;
- 2 *Initialize the target network*, $\theta' = \theta$;
- 3 *Initialize the experience replay buffer* D ;
- 4 **for** $Episode = 1$ to $Episode_{max}$ **do**
- 5 initial state $s = 0$;
- 6 **for** $step = 1$ to $step_{max}$ **do**
- 7 Select action a based on the current network $Q(s, a)$ using an ε -greedy strategy;
- 8 $CW_{thr} = 2^7 \times (1 + a)$;
- 9 apply CW_{thr} to obtain reward r and new observation o , with probability $O(o | s', a)$ the new state s' obtained;
- 10 Store (s, a, r, s') in the replay buffer D ;
- 11 **if** there are enough data in D **then**
- 12 **for** $i = 1$ to N **do**
- 13 extract Samples (s_i, a_i, r_i, s'_i) ;
- 14 compute $y_i = r_i + \gamma \times \max_{a'} Q(s'_i, a')$;
- 15 **end for**
- 16 $L(\theta) = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i))^2$ minimize the target loss function;
- 17 Update the current network $Q(s, a; \theta)$;
- 18 **If** $step \% C == 0$ **then**;
- 19 Update the target network: $Q(s, a; \theta)$;
- 20 **end if**
- 21 **end for**
- 22 **end for**

probability using the observation o , and stores the experience samples (s, a, r, s') for this period into the experience buffer. A small batch of samples d is also proposed from the experience buffer D , and the time-differential objective for each sample is computed. The parameter θ in the loss function $L(\theta)$ is updated using gradient descent. The target network is then updated, keeping the parameters of the evaluation network and the target network consistent at every C step. When the training phase is complete, the second phase, the evaluation phase, begins, and the agent only needs to observe the state and get the action through the trained model; there is no need for rewards since the agent body is already considered fully trained.

2) *Implementation of the Channel Access Process*: In the process of channel access, the AP is primarily responsible for centralized control. It periodically broadcasts CW_{thr} to all connected stations, evaluates the network performance of

Algorithm 2: AP Centralized Control Algorithm

Input : Define N_t : as the total number of frames sent
 N_r : as the number of frames received correctly.;
 Define V : as the transmission rate of the channel.;
 Define t : as the current slot time elapsed, T_{period} as the total slot time in one transmission period.;
 Define CW_{thr} : as contention window threshold value.;
 Define $H(P_{col})$ the historical collision rate, and TH as the throughput.;

- 1 **Initialize** N_t, N_r, t to 0;
- 2 AP generate CW_{thr} and broadcasts it;
- 3 AP listens to the channel;
- 4 **While** $t == T_{period}$ **do**
- 5 AP counts total number of frames sent N_t ;
- 6 AP counts number of frames received correctly N_r ;
- 7 $P_{col} = (N_t - N_r) / N_t$;
- 8 $H(P_{col}) = P_{col} - \text{Last}(P_{col})$;
- 9 $TH = \text{bit}(N_r) / (V \times \text{Sec}(T_{period}))$;
- 10 $H(P_{col})$ as observation o , TH as reward r both sent to Algorithm 1;
- 11 $t = 0$;
- 12 **end While**

the current CW_{thr} within that time period, and calculates the normalized throughput TH and historical collision.

rate $H(P_{col})$. Subsequently, these pieces of information are transmitted to the reinforcement learning process to regenerate a new CW_{thr} . Then, the AP and stations undergo transmission processes again, initiating the next transmission period. Through continuous iteration and optimization, the reinforcement learning process gradually learns the optimal CW_{thr} , thereby enhancing the overall network performance.

The details of the AP centralized control process are shown in Algorithm 2. Firstly, The AP generates a CW_{thr} value based on the action selected during the reinforcement learning process and broadcasts it to all connected stations. Subsequently, within one transmission period, the AP obtains the total number of successfully transmitted frames (N_r) by counting the number of ACK frames, and the total number of sent frames (N_t) by counting the number of frames sent by stations. At the end of the transmission period, the collision rate P_{col} of this CW_{thr} is calculated using (19). The historical collision rate $H(P_{col})$ is calculated as the difference between the collision rate of the current period, P_{col} , and the collision rate of the previous transmission period, $\text{Last}(P_{col})$. The calculation formula for the standardized throughput (TH) is shown in (23), where TH represents the network throughput, defined as the ratio of successfully received bits to the maximum expected bits. $\text{bit}(N_r)$ is the number of successfully received bits, V is the transmission rate of the channel, and T_{period} is the total slot time in one transmission period, Sec represents converting the corresponding slot time into seconds. $H(P_{col})$ as observation, TH as reward r , both sent to Algorithm 1 for predicting and adjusting the next CW_{thr} .

$$TH = \frac{bit(N_r)}{(V \times Sec(T_{period}))} \quad (23)$$

The station is responsible for selecting an appropriate adjustment strategy based on the CW_{thr} broadcasted by the AP to optimize its contention window value, CW_{cur} . The station compares CW_{cur} with the CW_{thr} when it needs to change the contention window after a data frame transmission. If $CW_{cur} \leq CW_{thr}$, it indicates that there are fewer competing stations in the current network and the load is light. STAs implement linear increase and exponential decrease strategies to reduce the waiting delay. If $CW_{cur} > CW_{thr}$, it indicates that there are many competing stations and a heavy network load on the current network. STAs implement exponential increase and linear decrease strategies to reduce the probability of collisions. This adaptive CW adjustment strategy allows stations to dynamically adjust their CW based on current network conditions, thereby reducing collision rates, increasing throughput, and optimizing overall Wi-Fi network performance.

The details of the channel access process for STAs are shown in Algorithm 3. First, STAs obtain the CW_{thr} from the beacon frame sent by AP. When the channel is idle for a consecutive DIFS time, STAs start the backoff process and randomly select a value from the current CW range, which is BT. The station's BT value decreases by one in every idle slot on the channel. If the station's BT value is zero, the channel is occupied, and a data frame is sent. Then, according to the success or failure of the STAs transmission, combined with the CW threshold, different CW optimization strategies are adopted. If $CW_{cur} \leq CW_{thr}$, Strategy 1 is adopted and uses the linear increase and exponential decrease strategy to adjust the station's CW_{cur} . If $CW_{cur} > CW_{thr}$, Strategy 2 is adopted and uses the exponential increase and linear decrease strategy to adjust the station's CW_{cur} . The CW_{cur} needs to be confirmed after adjustment, and if it exceeds the specified range of IEEE802.11, it needs to be adjusted. Finally, If the station's data frame is transmitted successfully, the station will wait for the DIFS idle time, then regenerate the BT value to contend for the channel and proceed with data transmission, until all the data that needs to be sent by the station is transmitted.

V. PERFORMANCE EVALUATION

A. Simulation Parameter Setting

In this article, we implemented the ACWB-DQN algorithm using the Pral and PaddlePaddle reinforcement learning frameworks provided by Baidu and the Python programming language. To evaluate the performance of the algorithm, we simulate the CSMA/CA protocol in IEEE 802.11 for medium access control (MAC) and compare the ACWB-DQN algorithm with the traditional CSMA/CA, deep reinforcement learning-based CCOD-DQN, and SETL-DQN algorithms.

The network scenario of the paper is shown in Fig. 4. This experiment uses the IEEE 802.11ax standard with 1024-QAM modulation, 5/6 coding rate, 20 MHz channel, single AP and multi-user transmission, and an 11 Mbps transmission rate

Algorithm 3: Channel Access for STAs Algorithm

Define CW_{cur} as the contention window value, CW_{thr} contention window threshold value, CW_{max} as the maximum contention window value, CW_{min} as the minimum contention window value. Define BT as the backoff timer value. Define i as the index number of the station.

```

1 Initialize  $CW_{cur}[i]$  to  $CW_{min}$ ;
2 STA  $[i]$  receives the  $CW_{thr}$ ;
3 STA  $[i]$  listens to the channel;
4 While the channel is idle for a DIFS time do
5    $BT[i] \leftarrow \text{random}(0, CW_{cur}[i] - 1)$ ;
6   While  $BT[i] \neq 0$  do
7     if the channel is idle for a slot time then
8        $BT[i] = BT[i] - 1$ ;
9     else:
10       $BT[i] = BT[i]$ ;
11    end if
12  end While
13  the station  $i$  starts occupying the channel and
  transmitting data frame;
14  if data frame is sent successfully then
15    if  $CW_{cur}[i] > CW_{thr}$  then
16       $CW_{cur}[i] = CW_{cur}[i] - 32$ ;
17    else:
18       $CW_{cur}[i] = CW_{cur}[i]/2$ ;
19    end if
20  else:
21    if  $CW_{cur}[i] > CW_{thr}$  then
22       $CW_{cur}[i] = CW_{cur}[i] \times 2$ ;
23    else:
24       $CW_{cur}[i] = CW_{cur}[i] + 32$ ;
25    end if
26  end if
27  if  $CW_{cur}[i] > CW_{max}$  then
28     $CW_{cur}[i] = CW_{max}$ ;
29  end if
30  if  $CW_{cur}[i] < CW_{min}$  then
31     $CW_{cur}[i] = CW_{min}$ ;
32  end if
33  If the station  $i$  has finished transmitting data then
    break;
34 end While

```

from station to AP. In addition, it is assumed that (1) the AP can be aware of the current network status (i.e., collision rate and throughput of the current network). (2) Each station can adjust its own CW value. (3) The AP periodically broadcasts the CW threshold via beacon frames.

The experimental environment is as follows: CPU: Intel Core i5-9500 3.00GHz, RAM: 16.00GB on which the Python development environment is configured and Scikit-learn and Scipy libraries are installed. The Python interpreter version is Python 3.6, the Pral version is 1.4.3, and the PaddlePaddle version is 1.8.5. The key parameters in the experiment are shown in Table I. The network used by the algorithm is a

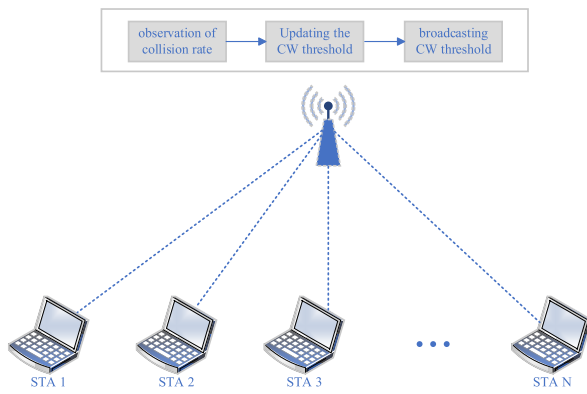


Fig. 4. Network scenario diagram.

TABLE I
EXPERIMENT-SPECIFIC PARAMETER SETTINGS

Reinforcement of The Reinforcement Learning Process		The Channel Access Process	
parameters	value	parameters	value
Learn frequency	5	CW_{min}	15
Memory size	20,000	CW_{max}	1023
Batch size	32	Slot	9us
Learning rate α	0.001	SIFS	16us
Epsilon greedy ϵ	0.1~1	DIFS	34us
Epsilon greedy ϵ decrement	0.000001	Data frame size	8184bit
Step _{max}	200	Channel bit rate	11Mbps
Episode _{max}	20		

DNN of three hidden-layer fully connected networks with 128 output dimensions and uses Relu as the activation function.

B. Simulation Experiments and Analysis

In the simulation experiments, it is assumed that there are no hidden nodes in the current network transmission, and only when nodes collide will it lead to data frame transmission failure. First, the first experiment tests the ACWB-DQN algorithm's CW thresholds at different station densities. The number of competing stations ranges from 10 to 150 at intervals of 20 stations, and the learning phase is run for 20 rounds to converge to a stable value. The experimental results are shown in Fig. 5. When the number of stations is 10, the CW threshold value is 1024, CW threshold decreases linearly with the increase in the number of competing stations, and when the number of competing stations is 150, the CW threshold value is 128. The CW threshold value is dynamically adjusted according to the current collision rate and throughput of the network, which is designed to adapt to different network situations. When the number of competing stations is small, the network load is light. In this case, the CW threshold value is kept at a larger value to ensure that it is as large as possible compared to the CW value of the stations, thus implementing strategy 1 which uses a linear increase and exponential decrease strategy to maintain the CW value of the stations smaller to minimize the waiting delay of the frames

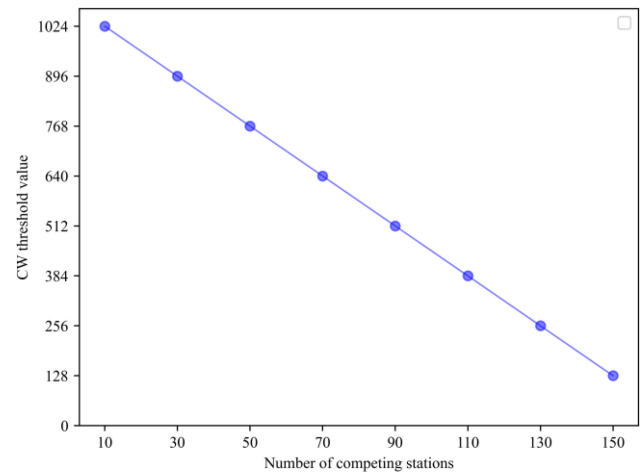


Fig. 5. CW thresholds for different numbers of competing stations.

during the transmission process. On the other hand, when the number of competing stations is large, the network load becomes heavier. To try to avoid collision between the stations, the CW threshold value is kept small to ensure that it is as small as possible compared to the CW value of the stations, thus implementing strategy 2 which employs an exponential increase and linear decrease strategy to maintain the CW value of the stations larger to avoid collision between the stations.

As for collision rates, the paper [14] analyzes and compare the SETL-DQN scheme it proposed with the CCOD-DQN scheme, traditional CSMA and SETL schemes. As the number of competing stations increases, the collision rate also rises. In the comparison, both the SETL-DQN and CCOD schemes exhibit significantly lower collision rates compared to traditional CSMA and SETL schemes. This is attributed to the adoption of deep reinforcement learning, which allows for easier adjustment of CW based on network conditions, rather than relying on multiple collisions to find a suitable CW, thus effectively reducing collision rates. In the comparison between the CCOD-DQN and SETL-DQN schemes, the collision rate of the SETL-DQN scheme is slightly lower than that of the CCOD-DQN scheme. This is because, in scenarios with a high number of stations, the CCOD-DQN scheme can rapidly reduce collision rates by directly adjusting the CW to CW_{max} . However, this approach leads to a sharp increase in waiting delay. The ACWB-DQN algorithm proposed in this article adopts different strategies for adjusting the CW size based on the number of competing stations and the network load. In scenarios with fewer competing stations and low network load, the algorithm gradually increases the CW size to reduce collision rates while minimizing waiting delay. In contrast, in scenarios with a high number of competing stations and high network load, the algorithm does not directly set CW to CW_{max} to minimize collision rates like the CCOD-DQN method. Instead, the algorithm utilizes an exponential growth method to adjust the contention window, thereby balancing the reduction of collision rates and the sharp increase in waiting delay.

The second experiment tested the collision rate of traditional CSMA, deep learning-based CCOD-DQN, SETL-DQN, and

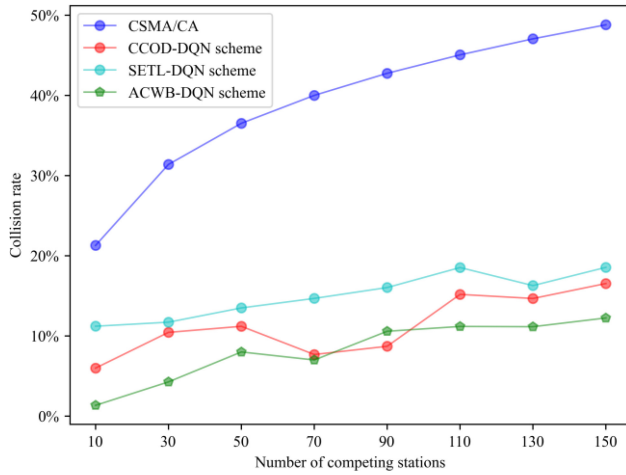


Fig. 6. Comparison of collision rates for different schemes.

the proposed ACWB-DQN scheme under different station densities. In Fig. 6, the collision rate of the CW optimization scheme based on deep reinforcement learning is obviously lower than that of traditional CSMA/CA solutions. In the case of low station density (10 stations), the collision rate of the CSMA/CA scheme, the CCOD-DQN scheme and the SETL-DQN scheme are 21.28%, 5.98%, 11.21%, respectively. The collision rate of the proposed ACWB-DQN scheme is only 1.13% when the station is 10. That collision rate is 20.15% lower than that of the CSMA/CA scheme, 4.85% lower than that of the CCOD-DQN scheme, and 10.08% lower than that of the SETL-DQN scheme. In the case of high station density (150 stations), the collision rate of each scheme has increased to a certain extent; the collision rate of the CSMA/CA scheme is 48.79%, the collision rate of the CCOD-DQN scheme is 16.53%, and the collision rate of the SETL-DQN scheme is 18.54%. When the number of stations is 150, the collision rate of the ACWB-DQN proposed in this article is 4.24%, which is 44.55% lower than that of the CSMA/CA scheme, 12.29% lower than that of the CCOD-DQN scheme, and 14.32% lower than that of the SETL-DQN scheme. In addition, the ACWB-DQN scheme proposed in this article has a collision rate of less than 5% under different station densities (the number of stations ranges from 10 to 150). Therefore, the ACWB-DQN scheme proposed in this article can effectively reduce the collision rate of the network with different station densities. The scheme still maintains a lower collision rate in the network with high station densities.

The paper [12] compares and analyzes the proposed CCOD-DQN algorithm with the standard IEEE 802.11 mechanism. The findings show that under both static and dynamic competition scenarios, the CCOD-DQN scheme significantly reduces throughput degradation caused by an increase in the number of stations with the help of DRL technology. The paper [14] provides a comparative analysis between the proposed SETL-DQN and CCOD-DQN, and CSMA and SETL, demonstrating that DRL-based solutions achieve significantly higher throughput compared to traditional CSMA and SETL approaches. For SETL-DQN and CCOD-DQN schemes, CCOD maintains better throughput under light loads. However, under heavy

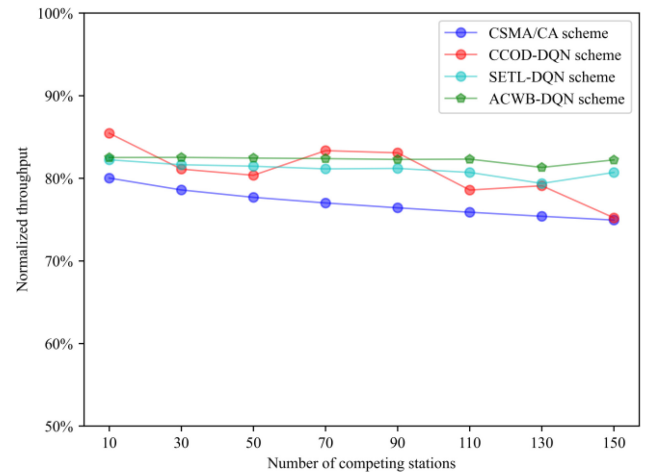


Fig. 7. Normalized throughput comparison of different schemes.

loads, due to its backoff algorithm flaw, CCOD-DQN resets the CW value to CW_{min} after a successful data frame transmission, severely impacting throughput performance. On the other hand, the SETL-DQN scheme maintains stable throughput under both light and heavy load network conditions. The ACWB-DQN algorithm proposed in this article decreases the CW value exponentially after a successful data transmission under light loads, swiftly reducing station waiting delays and further enhancing its transmission rate. Different from the linear decrease in SETL-DQN, under heavy loads, successful data transmission with ACWB-DQN results in a slow linear decrease in CW values, which prevents collision rates from increasing caused by rapid CW reduction. Consequently, network throughput is improved.

The third experiment tests the normalized throughput of different schemes, and the results of the experiment are shown in Fig. 7. It shows that the deep reinforcement learning-based CCOD-DQN, SETL-DQN, and the ACWB-DQN scheme proposed in this article improve the throughput compared with the traditional CSMA/CA. The ACWB-DQN schemes proposed in this article and SETL-DQN have more stable normalized throughputs at different station densities and still maintain high normalized throughput even in high-density networks. The ACWB-DQN scheme has a station throughput of 82.23% of the expected maximum throughput at higher station densities (the number of stations is 150), the SETL-DQN scheme achieved 80.715% compared to 74.945% for the CSMA/CA scheme, while the CCOD-DQN scheme reached 75.214%. It improves the normalized throughput by 7.285% over the CSMA/CA scheme, 1.515% over the SETL-DQN scheme, and 7.016% compared with the CCOD-DQN scheme. Therefore, the ACWB-DQN scheme proposed in this article can effectively improve the throughput under different station densities; even in cases of higher station densities, the scheme can still maintain a high and stable throughput.

Finally, the traditional CSMA, deep learning-based CCOD-DQN, SETL-DQN, and ACWB-DQN are compared and analyzed, as shown in Table II. CSMA/CA scheme: The BEB algorithm is used to adjust the CW, which can effectively reduce the occurrence of collisions when the network scale is

TABLE II
COMPARATIVE ANALYSIS OF DIFFERENT SCHEMES

Scheme	Reinforcement learning	Contention window adjustment strategy	Collision rate	Throughput capacity	Adaptability
CSMA/CA	×	BEB algorithm	With the increase of the number of stations, the collision rate is high	With the increase of the number of sites, the throughput is low	Only suitable for small and lightly loaded networks
CCOD-DQN [17]	✓	Based on CSMA/CA, DRL technology is used to predict and adjust the size of the contention window	Compared with CSMA scheme, The collision rate is reduced under different load degrees, and the collision rate is higher under high load degrees	Compared with CSMA scheme, the throughput under different load levels is improved	Adapt to the network conditions of different load levels, and can adapt to the change in network conditions
SETL-DQN [19]	✓	The DRL technology is used to predict and adjust the contention window threshold, and the SETL algorithm is used to adjust the contention window	Compared with CSMA scheme, The collision rate is reduced under different load degrees, and the collision rate is higher under high load degrees	Compared with CSMA scheme, it maintain stable throughput and improve throughput at different load levels	Adapt to the network conditions of different load levels, and can adapt to the change in network conditions
ACWB-DQN	✓	The DRL technology is used to predict and adjust the contention window threshold, and different adjustment strategies are set according to the sending situation and threshold of the station	Compared with the CSMA scheme and other schemes, the collision rate is effectively reduced under different load degrees, and the collision rate is low under high load degrees	Compared with the CSMA scheme and other schemes, it maintain stable throughput and improve throughput at different load levels	Adapt to the network conditions of different load levels, and can adapt to the change in network conditions

small and the load degree is light. However, with the increase in network scale and the increase in load degree, the collision rate will rise sharply, and the throughput will also decline. CCOD-DQN scheme: The DRL technology is used to predict the CW value, improve the throughput of the Wi-Fi network, and reduce the collision rate in the case of a heavily loaded network. However, due to the use of BEB to adjust and predict the CW value, the CW has been adjusted exponentially. Due to the flaws of the BEB algorithm employed, when a data frame transmission is successful, the contention window is reset to CW_{min} . Moreover, under high load conditions, DRL continuously resets the CW of the station to CW_{max} , resulting in an increase in transmission delay and a decrease in throughput. SETL-DQN scheme: By using DRL to explore the CW threshold, it is used to distinguish the load of the network. By using the SETL algorithm to adjust the CW value, the exponential adjustment strategy is used when the network load is light, and the linear adjustment strategy is used when the network load is heavy. In the SETL-DQN scheme the network conditions are distinguished and the CW adjustment strategy is improved, the collision rate in the network is further reduced and the throughput is improved, so the collision is less likely under the lightly loaded network. The exponential adjustment may increase the waiting delay at the station. In a network with a heavy load, the collision is likely to be higher, and the linear adjustment increases the CW range by a small amount, increasing the collision probability. The ACWB-DQN scheme proposed in this article: By using DRL to explore the CW threshold suitable for the current network environment and dynamically adjust the threshold according to the changes in throughput and collision rate in the network, different CW optimization strategies are adopted according to the transmission situation of the station and the current CW threshold of the network to reduce the waiting delay

and improve the transmission efficiency when the network is lightly loaded. When the network is heavily load, the collision probability is effectively reduced, the throughput of the network is improved, and the balance between throughput and transmission delay is better realized, thus improving the overall network performance.

VI. CONCLUSION

This article proposes an ACWB-DQN scheme for optimizing the contention window, explores a CW threshold suitable for the current network environment by using DRL technology, and dynamically adjusts the CW threshold according to the changes in throughput and collision rate in the network. On this basis, according to the current CW threshold and the station transmission situation, the appropriate CW optimization strategy is adopted to adjust the contention window of the station. Finally, through the experimental verification analysis, the scheme can dynamically adapt to different network conditions, effectively reduce the collision rate of stations in high-density network environments, maintain stable throughput, and significantly improve the performance of the Wi-Fi network.

REFERENCES

- [1] R. Nazir, A. A. Laghari, K. Kumar, S. David, and M. Ali, "Survey on wireless network security," *Arch. Comput. Methods Eng.*, vol. 29, no. 3, pp. 1591–1610, May 2022.
- [2] E. Mozaffariahrar, F. Theoleyre, and M. Menth, "A survey of Wi-Fi 6: Technologies, advances, and challenges," *Future Internet*, vol. 14, no. 10, p. 293, Oct. 2022.
- [3] Md. I. Hussain, N. Ahmed, Md. Z. I. Ahmed, and N. Sarma, "QoS provisioning in wireless mesh networks: A survey," *Wireless Pers. Commun.*, vol. 122, no. 1, pp. 157–195, Jan. 2022.
- [4] H. F. Sultan and M. N. Fadhl, "Wireless sensor networks designs and protocols: A survey," in *Proc. 8th Int. Conf. Contemp. Inf. Technol. Math. (ICCTIM)*, Mosul, Iraq, Aug. 2022, pp. 318–323.

- [5] S. K. Memon et al., "A survey on 802.11 MAC industrial standards, architecture, security & supporting emergency traffic: Future directions," *J. Ind. Inf. Integr.*, vol. 24, Dec. 2021, Art. no. 100225.
- [6] S. Szott et al., "Wi-Fi meets ML: A survey on improving IEEE 802.11 performance with machine learning," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 3, pp. 1843–1893, 3rd Quart., 2022.
- [7] M. M. Afsar, T. Crump, and B. Far, "Reinforcement learning based recommender systems: A survey," 2022, *arXiv:2101.06286*.
- [8] J. Lansky, A. M. Rahmani, and M. Hosseinzadeh, "Reinforcement learning-based routing protocols in vehicular Ad Hoc networks for intelligent transport system (ITS): A survey," *Mathematics*, vol. 10, no. 24, p. 4673, Dec. 2022.
- [9] P. Upadhyay et al., "An improved deep reinforcement learning routing technique for collision-free VANET," *Sci. Rep.*, vol. 13, no. 1, Dec. 2023, Art. no. 21796.
- [10] X. Liu et al., "A reinforcement learning-based congestion control approach for V2V communication in VANET," *Applied Sciences*, vol. 13, no. 6, p. 3640, Mar. 2023.
- [11] B. Liu, G. Xu, G. Xu, C. Wang, and P. Zuo, "Deep reinforcement learning-based intelligent security forwarding strategy for VANET," *Sensors*, vol. 23, no. 3, p. 1204, Jan. 2023.
- [12] W. Sun, L. Zhang, Q. Lv, Z. Liu, W. Li, and Q. Li, "Dynamic collaborative optimization of end-to-end delay and power consumption in wireless sensor networks for smart distribution grids," *Comput. Commun.*, vol. 202, pp. 87–96, Mar. 2023.
- [13] X. Wang, H. Chen, and S. Li, "A reinforcement learning-based sleep scheduling algorithm for compressive data gathering in wireless sensor networks," *J. Wireless Commun. Network.*, vol. 2023, no. 1, p. 28, Mar. 2023.
- [14] O. Iacoboaiea, J. Krolkowski, Z. B. Houidi, and D. Rossi, "From design to deployment of zero touch deep reinforcement learning WLANs," *IEEE Commun. Mag.*, vol. 61, no. 2, pp. 104–109, Feb. 2023.
- [15] F. Frommel, G. Capdehourat, and F. Larroca, "Reinforcement learning based coexistence in mixed 802.11ax and legacy WLANs," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Glasgow, U.K., Mar. 2023, pp. 1–6.
- [16] S. Balhara et al., "A survey on deep reinforcement learning architectures, applications and emerging trends," *IET Commun.*, vol. 19, Jul. 2022, Art. no. 12447.
- [17] R. Ali, N. Shahin, Y. T. Kim, B. S. Kim, and S. W. Kim, "Channel observation-based scaled backoff mechanism for high-efficiency WLANs," *Electron. Lett.*, vol. 54, no. 10, pp. 663–665, May 2018.
- [18] X. Cai et al., "Fractional backoff algorithm for the next generation WLAN," in *Proc. 11th EAI Int. Conf. Wireless Internet (WiCON)*, Springer, Oct. 2018, pp. 24–31.
- [19] R. Ali, N. Shahin, Y. B. Zikria, B.-S. Kim, and S. W. Kim, "Deep reinforcement learning paradigm for performance optimization of channel observation-based MAC protocols in dense WLANs," *IEEE Access*, vol. 7, pp. 3500–3511, 2022.
- [20] R. Ali, A. Nauman, Y. B. Zikria, B. S. Kim, and S. W. Kim, "Performance optimization of QoS-supported dense WLANs using machine-learning-enabled enhanced distributed channel access (MEDCA) mechanism," *Neural Comput. Appl.*, vol. 32, no. 17, pp. 13107–13115, Dec. 2020.
- [21] W. Wydmanski and S. Szott, "Contention window optimization in IEEE 802.11ax networks with deep reinforcement learning," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Mar. 2021, pp. 1–6.
- [22] C.-H. Ke, C.-C. Wei, K. W. Lin, and J.-W. Ding, "A smart exponential-threshold-linear backoff mechanism for IEEE 802.11 WLANs," *Int. J. Commun. Syst.*, vol. 24, no. 8, pp. 1033–1048, Aug. 2011.
- [23] C.-H. Ke and L. Astuti, "Applying deep reinforcement learning to improve throughput and reduce collision rate in IEEE 802.11 networks," *KSII TIS*, vol. 16, no. 1, pp. 334–349, Jan. 2022.
- [24] G. Bianchi, "Performance analysis of the IEEE 802.11 distributed coordination function," *IEEE J. Sel. Areas Commun.*, vol. 18, no. 3, pp. 535–547, Mar. 2000.
- [25] A. Feriani and E. Hossain, "Single and multi-agent deep reinforcement learning for AI-enabled wireless networks: A tutorial," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 2, pp. 1226–1252, 2nd Quart., 2021.
- [26] X. Wang et al., "Deep reinforcement learning: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 5064–5078, Apr. 2024.
- [27] S. E. Li, "Deep reinforcement learning," in *Reinforcement Learning for Sequential Decision and Optimal Control*. Singapore: Springer, Apr. 2023, pp. 365–402.



Zhi-Bin Zuo was born in Henan, China, in 1979. He received the Ph.D. degree from the School of Electronic Technology, Information Engineering University, in 2020. He is currently a Lecture with the College of Information Science and Engineering, Henan University of Technology. His main research interests include SDN and network security.



De-Min Wang was born in Henan, China, in 1998. He received the B.S. degree from the School of Software, Zhengzhou University of Light Industry, in 2022. He is currently pursuing the master's degree with the College of Information Science and Engineering, Henan University of Technology. His main research interests include SDN and network security.



Mi-Mi Ma was born in Henan, China, in 1987. She received the Ph.D. degree in applied mathematics from the School of Mathematics and Statistics, Wuhan University, in 2018. She is currently an Associate Professor with the College of Information Science and Engineering, Henan University of Technology. Her main research interests include number theory and cryptography.



Miao-Lei Deng was born in Henan, China, in 1977. He received the Ph.D. degree from Xidian University, Xi'an China, in 2010. He is currently a Professor with the Henan University of Technology, Zhengzhou, China. His current research interests include information security and cryptographic protocol design. He is a Senior Member of CCF.



Chun Wang was born in Henan, China, in 2000. He received the B.S. degree from the School of Mechanical and Electrical Information, Shangqiu University, in 2022. He is currently pursuing the master's degree with the College of Information Science and Engineering, Henan University of Technology. His main research interests include SDN and network security.